Sprint 4: Modelat SQL

Nivell 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

Antes de empezar a realizar los ejercicios siguientes, comenzamos con crear la base de datos, llamada **operations.**

```
/*Nivell 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui,
almenys 4 taules de les quals puguis realitzar les següents consultes:*/

/*Antes de empezar a realizar los ejercicios empezamos por Crear la base de datos y luego sus tablas, utilizando los archivos adjuntos dejados en la plataforma.
*/
CREATE DATABASE operations; #Escogemos un nombre que tenga relacion y sentido junto a los datos a utilizar
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	17:07:58	CREATE DATABASE operations	1 row(s) affected	0.016 sec

Al revisar los archivos CSV, hemos notado que hay 2 archivos con información sobre usuarios, uno para **europa** y otro para **américa**, por lo que en vez de crear
2 tablas de dimensiones de usuarios, tendremos una que contenga a todos, uniendolas y agregando una columna identificativa del continente llamada **region** para asi mantener
la integridad de los datos y no perder la utilidad del continente.

Procedemos a crear una tabla en crudo definida como **users_raw** y otra definida como **users** donde se le introducirán los valores limpios luego de transformarlos.

```
#Al revisar los archivos CSV, hemos notado que hay 2 archivos con informacion sobre usuarios, uno para europa y otro para america, por lo que en vez de crear
# 2 tablas de dimensiones de usuarios, tendremos una que contenga a todos, uniendolas y agregando una columna identificativa del continente llamada region para asi mantener
#la integridad de los datos y no perder la utilidad del continente.
#Se crearan 2 tablas user, una en crudo definida como users_raw y otra definida como users donde se le introduciran los valores limpios luego de transformarlos

CREATE TABLE IF NOT EXISTS `users_raw` (  -- Usaremos esta tabla para introducir los datos del archivo CSV sin transformacion ni limpieza previa
  `id` int DEFAULT NULL,
  `name` varchar(100) DEFAULT NULL,
  `surname` varchar(100) DEFAULT NULL,
  `phone` varchar(150) DEFAULT NULL,
  `email` varchar(150) DEFAULT NULL,
  `birth_date` varchar(50) DEFAULT NULL,    -- Aqui podemos definir la columna birth_date como varchar para transformar y luego pasar a DATE
  `country` varchar(150) DEFAULT NULL,
  `city` varchar(150) DEFAULT NULL,
  `postal_code` varchar(20) DEFAULT NULL,
  `address` varchar(255) DEFAULT NULL
);

CREATE TABLE IF NOT EXISTS `users` (
  `id` int NOT NULL,
  `name` varchar(100) DEFAULT NULL,
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:43:09	CREATE TABLE IF NOT EXISTS `users_raw` (-- Usaremos esta tabla para introducir los datos del archivo CSV s...	0 row(s) affected	0.062 sec

Procedemos a crear la tabla **users**, definiendo los atributos con sus tipo de dato correcto listos para recibir a través de un SELECT los datos de la tabla **users_raw**

```
32  CREATE TABLE IF NOT EXISTS `users` (
33    `id` int NOT NULL,
34    `name` varchar(100) DEFAULT NULL,
35    `surname` varchar(100) DEFAULT NULL,
36    `phone` varchar(150) DEFAULT NULL,
37    `email` varchar(150) DEFAULT NULL,
38    `birth_date` DATE DEFAULT NULL,
39    `region` varchar (50) DEFAULT NULL,  #Columna definida para no perder la granularidad de los datos al juntar ambos archivos CSV de users
40    `country` varchar(150) DEFAULT NULL,
41    `city` varchar(150) DEFAULT NULL,
42    `postal_code` varchar(20) DEFAULT NULL,
43    `address` varchar(255) DEFAULT NULL,
44    PRIMARY KEY (`id`)
45  ) ;
46
47
48    #Creamos la tabla credit_cards con los campos segun el archivo csv, asignando a cada columna su tipo de dato y longitud
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:47:06	CREATE TABLE IF NOT EXISTS `users` (`id` int NOT NULL, `name` varchar(100) DEFAULT NULL, `surnam...	0 row(s) affected	0.031 sec

Creamos la tabla **credit_cards_raw** para obtener todos los datos en crudo del archivo CSV para luego transformar

```
48    #Creamos la tabla credit_cards con los campos segun el archivo csv, asignando a cada columna su tipo de dato y longitud.
49    #Ademas de definir la primary key definimos tambien la foreign key que relaciona la tabla credit_cards con users,
50    # Definimos un indice llamado idx_user_id para optimizar las consultas.
51
52    #creamos la tabla credit_cards_raw para obtener todos los datos en crudo del archivo CSV para luego transformar
53  CREATE TABLE IF NOT EXISTS `credit_cards_raw`  (
54    `id` varchar(20) ,
55    `user_id` int DEFAULT NULL,
56    `iban` varchar(50) DEFAULT NULL,
57    `pan` varchar(20) DEFAULT NULL,
58    `pin` varchar(4) DEFAULT NULL,
59    `cvv` varchar(4) DEFAULT NULL,
60    `track1` varchar(100) DEFAULT NULL,
61    `track2` varchar(100) DEFAULT NULL,
62    `expiring_date` varchar(50) DEFAULT NULL
63  ) ;
64
65    #Creamos la tabla credit_cards asignando su respectivo atributo a cada columna, y definiendo la primary key, foreign key y un indice.
66  CREATE TABLE IF NOT EXISTS `credit_cards`  (
67    `id` varchar(20) NOT NULL,
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:10:19	CREATE TABLE IF NOT EXISTS `credit_cards_raw` (`id` varchar(20) , `user_id` int DEFAULT NULL, `iban` ...	0 row(s) affected	0.031 sec

Creamos la tabla **credit_cards** asignando a cada columna su tipo de dato y longitud.
Además de definir la primary key definimos también la foreign key que relaciona la tabla **credit_cards** con **users**,
Definimos un índice asignado a **user_id** llamado **idx_user_id** para optimizar las consultas

```sql
#Creamos la tabla credit_cards asignando su respectivo atributo a cada columna, y definiendo la primary key, foreign key y un indice.
CREATE TABLE IF NOT EXISTS `credit_cards`  (
  `id` varchar(20) NOT NULL,
  `user_id` int DEFAULT NULL,
  `iban` varchar(50) DEFAULT NULL,
  `pan` varchar(20) DEFAULT NULL,
  `pin` varchar(4) DEFAULT NULL,
  `cvv` varchar(4) DEFAULT NULL,
  `track1` varchar(100) DEFAULT NULL,
  `track2` varchar(100) DEFAULT NULL,
  `expiring_date` DATE DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_user_id` (`user_id`),
  CONSTRAINT `fk_user_id` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
) ;


#creando la tabla companies
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:14:36	CREATE TABLE IF NOT EXISTS `credit_cards` (`id` varchar(20) NOT NULL, `user_id` int DEFAULT NULL, ...	0 row(s) affected	0.047 sec

Creamos la tabla **companies**, no hace falta en este caso crear una tabla en crudo ya que no es necesario hacer transformaciones de datos porque ya están limpios.

```sql

#Creando la tabla companies, no hace falta en este caso creaer una tabla en crudo ya que es necesario hacer transformaciones de datos porque en este caso ya estan limpios

CREATE TABLE IF NOT EXISTS `companies` (
  `id` varchar(20) NOT NULL, #Tener en cuenta el nombre del id en el archivo CSV que es company_id
  `company_name` varchar(255) DEFAULT NULL,
  `phone` varchar(15) DEFAULT NULL,
  `email` varchar(100) DEFAULT NULL,
  `country` varchar(100) DEFAULT NULL,
  `website` varchar(260) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ;



#creando la tabla products
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	09:58:55	CREATE TABLE IF NOT EXISTS `companies` (`id` varchar(20) NOT NULL, #Tener en cuenta el nombre del id ...	0 row(s) affected	0.047 sec

Para crear la columna **products**, se ha detectado que en la columna price se encuentra el símbolo "$" por lo que será necesario transformar esa columna
Creamos una tabla llamada **products_raw** que reciba los datos sin alterar del archivo csv:

```sql
-- Para crear la columna products, se ha detectado que en la columna price se encuentra el símbolo "$" por lo que será necesario transformar esa columna
-- Creamos una tabla llamada products_raw que reciba los datos sin alterar del archivo csv:

CREATE TABLE IF NOT EXISTS `products_raw`(
`id` int,
`product_name` varchar (100) DEFAULT NULL,
`price` varchar(50) DEFAULT NULL , -- Asignamos como varchar y luego se procede a transformar cuando se inserte en la otra tabla
`colour` varchar (20) DEFAULT NULL,
`weight` DECIMAL(10,2) DEFAULT NULL,
`warehouse_id` varchar(20) DEFAULT NULL
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:12:50	CREATE TABLE IF NOT EXISTS `products_raw`(`id` int, `product_name` varchar (100) DEFAULT NULL, `price` ...	0 row(s) affected	0.047 sec

Creamos la tabla **products**, que contendrá los valores en limpio y con su respectiva primary key

```sql
-- Creamos la tabla products, que contendrá los valores en limpio y con su respectiva primary key.

CREATE TABLE IF NOT EXISTS `products`(
`id` int NOT NULL PRIMARY KEY ,
`product_name` varchar (100) DEFAULT NULL,
`price` DECIMAL(10,2) DEFAULT NULL ,
`colour` varchar (20) DEFAULT NULL,
`weight` DECIMAL(10,2) DEFAULT NULL,
`warehouse_id` varchar(20) DEFAULT NULL
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:13:28	CREATE TABLE IF NOT EXISTS `products`(`id` int NOT NULL PRIMARY KEY , `product_name` varchar (100) D...	0 row(s) affected	0.047 sec

Creando la tabla **transactions**, hemos notado que en el archivo CSV para la creación de la tabla **transactions**, existe una columna llamada product_ids en la cual tiene varios ids en una misma celda, por lo que procederemos a crear una tabla llamada **transaction_products**, donde podremos transformar los datos para evitar errores. Debido a que manipular la información donde hay más de un id en una misma celda puede darnos problemas y dificultades a la hora de filtrar o agrupar.

Creamos una tabla llamada **transactions_raw** donde insertamos los datos y luego procederemos a transformarlos, por lo que definiremos todas las columnas como **varchar**.

```sql
-- Creando la tabla transactions
-- Hemos notado que en el archivo CSV para la creacion de la tabla transactions, existe una columna llamada product_ids en la cual tiene varios ids en una misma celda
-- por lo que procederemos a crear una tabla llamada transaction_products, donde podremos transformar los datos para evitar errores
-- debido a que manipular la informacion donde hay mas de un id en una misma celda puede darnos problemas y dificultades a la hora de filtrar o agrupar.

-- Creamos una tabla llamada transactions_raw donde insertaremos los datos y luego procederemos a transformarlos, por lo que definiremos todas las columnas como varchar

CREATE TABLE `transactions_raw` (
  `id` VARCHAR(255),
  `card_id` VARCHAR(20),
  `business_id` VARCHAR(20),-- Esta columna actua como foreign key que hace referencia a la tabla companies
  `timestamp` VARCHAR(50),
  `amount` VARCHAR(20),
  `declined` VARCHAR(10),
  `product_ids` VARCHAR(255),
  `user_id` VARCHAR(20),
  `lat` VARCHAR(50),
  `longitude` VARCHAR(50)
  );
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:40:04	CREATE TABLE `transactions_raw` (`id` VARCHAR(255), `card_id` VARCHAR(20), `business_id` VARCHAR(...	0 row(s) affected	0.031 sec

Se crea la tabla **trancactions** la cual tendrá todos los datos transformados y limpios.
Se han creado 3 índices para optimizar las consultas, cada uno para las foreign key creadas: **company_id**(antes business_id), **user_id** y **card_id**.

```sql
#Creo la tabla trancactions con sus atributos
CREATE TABLE IF NOT EXISTS `transactions`(
 `id` varchar(255) NOT NULL,
  `card_id` varchar(20) DEFAULT NULL,
  `company_id` varchar(20) DEFAULT NULL,    -- Se renombra la columna `business_id` a `company_id` para mayor facilidad y congruencia en las consultas
  `timestamp` timestamp NULL DEFAULT NULL, -- timestamp que tiene el formato de fecha incorrecto ---> 12/12/2018 8:05 y lo transformaremos en ---> 2018-12-12 08:05:00
  `amount` DECIMAL(10,2) DEFAULT NULL,
  `declined` tinyint DEFAULT NULL,
  `user_id` int DEFAULT NULL,
  `lat` varchar(250) DEFAULT NULL,
  `longitude` varchar(250) DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_user_id` (`user_id`), -- Creamos Indices para user_id y card_id para tener las consultas optimizadas
  KEY `idx_card_id` (`card_id`),
  KEY `idx_company_id` (`company_id`),
  CONSTRAINT `fk_card_id` FOREIGN KEY (`card_id`) REFERENCES `credit_cards` (`id`),
  CONSTRAINT `fk_users_id` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`),
  CONSTRAINT `fk_company_id` FOREIGN KEY (`company_id`) REFERENCES `companies` (`id`)
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:48:04	CREATE TABLE IF NOT EXISTS `transactions`(`id` varchar(255) NOT NULL, `card_id` varchar(20) DEFAULT ...	0 row(s) affected	0.063 sec

Creamos la tabla puente llama **transaction_products** donde podremos de esta forma obtener un mejor modelado de datos y evitar errores por la columna **product_ids**

```sql
#Creamos la tabla puente donde podremos de esta forma obtener un mejor modelado de datos y evitar errores por la columna product_ids
CREATE TABLE IF NOT EXISTS `transaction_products`(
`transaction_id` varchar(255) NOT NULL,
`product_id` int NOT NULL,
PRIMARY KEY (`transaction_id`,`product_id`),
CONSTRAINT `fk_transaction_id` FOREIGN KEY (`transaction_id`) REFERENCES `transactions`(`id`),
CONSTRAINT `fk_product_id` FOREIGN KEY (`product_id`) REFERENCES `products`(`id`)

);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:51:33	CREATE TABLE IF NOT EXISTS `transaction_products`(`transaction_id` varchar(255) NOT NULL, `product_id` i...	0 row(s) affected	0.047 sec

Procedemos a importar e insertar los datos del archivo csv a **usars_raw**
primero insertamos el archivo CSV **american_users**

```sql
#Procedemos a importar e insertar los datos del archivo csv a usars_raw
#primero insertamos el archivo CSV american_users
LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\Sprint 4 MODELAT SQL\\Archivos adjuntos sprint 4\\american_users.csv'
INTO TABLE `users_raw`
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS ;

#Despues de ejecutarlo MySQL nos ha arrojado el Error 3948: Loading local data is disabled, por lo que debimos utilizar el comando  SET GLOBAL local_infile para activarlo
SET GLOBAL local_infile = 1;
#Ahora si pudimos ejercutar el LOAD DATA LOCAL INFILE arriba para ingresar los datos
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:49:46	LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\Sp...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.062 sec

Durante la carga de datos con el comando **LOAD DATA LOCAL INFILE** se presentaron errores relacionados a las configuraciones de seguridad del servidor y del cliente, además de la lectura de la ruta del archivo CSV. El primer error ha ocurrido porque no estaba activado el **`local_infile = ON`** para que acepte recibir archivos enviados por el programa, activandolo con el comando **SET GLOBAL local_infile=1;**

Workbench como cliente lo tiene desactivado por defecto por razones de seguridad por lo que hemos tenido que agregar el comando o el permiso **OPT_LOCAL_INFILE=1** en la configuración —> Database, Manage Server Connections, advanced y en others insertar: **OPT_LOCAL_INFILE=1** y luego reiniciar el cliente.

Una vez agregado hemos tenido que modificar los slash(\) de la dirección del archivo al intentar insertar nuevamente los datos del CSV, ya que el copiado desde la ruta de acceso falla,por lo que tuvimos que agregar un slash adicional para que el MySql sea capaz de leer la ruta y encontrar el archivo.

Gracias a estas configuraciones, fue posible ejecutar correctamente la carga de datos en la tabla **users_raw**.

Insertando el archivo CSV de **european_users** en la tabla **users_raw**

```
#Se procede a insertar el segundo archivo CSV a la tabla users_raw --> european_users
LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\Sprint 4 MODELAT SQL\\Archivos adjuntos sprint 4\\european_users.csv'
INTO TABLE `users_raw`
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS ;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:43:14	LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\Spr...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.141 sec

Antes de hacer la transformación y limpieza, revisamos la cantidad de países distintos en la tabla **users_raw** usando el comando SELECT.

—-> SELECT DISTINCT country FROM users_raw; Lo cual nos arroja 11 países distintos, con lo cual podremos hacer un Case cuando se haga la asignación de continente a la columna región en la tabla users.

Se procede a transformar y limpiar los datos de **users_raw** para insertarlos en la tabla **users**, aplicando transformación y limpieza al al atributo birth_date y a region.

```
-- Procedemos a transformar y limpiar los datos de users_raw para insertarlos en la tabla users

INSERT INTO `users`(id,name,surname,phone,email,birth_date,region,country,city,postal_code,address)
SELECT id,name,surname,phone,email,
STR_TO_DATE(birth_date,'%b %d, %Y'),
CASE -- Asignamos el valor de region con un CASE
    WHEN country in ('United States','Canada','Mexico') THEN 'American'
    WHEN country in ('United Kingdom','Spain', 'Netherlands','Sweden','France','Poland','Germany','Italy','Portugal') THEN 'European'
    ELSE 'Other'
 END,
country,city,postal_code,address
FROM users_raw;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:28:35	INSERT INTO `users`(id,name,surname,phone,email,birth_date,region,country,city,postal_code,address) SELECT...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.203 sec

Se procede a Insertar los datos del CSV a la tabla **credit_cards_raw** para luego transformarlos e insertarlos en la tabla **credit_cards**

```
#Procedemos a importar e insertar los datos del archivo csv a credit_cards_raw
LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\Sprint 4 MODELAT SQL\\Archivos adjuntos sprint 4\\credit_cards.csv'
INTO TABLE `credit_cards_raw`
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS ;

#Creamos la tabla credit_cards asignando su respectivo atributo a cada columna, y definiendo la primary key, foreign key y un indice
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:39:36	LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\Spr...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.156 sec

Se procede a transfromar los datos en el SELECT y a insertarlos en la tabla **credit_cards**
Se ha transformado la fecha por el formato erróneo en el que se encontraba

```
-- Se procede a transfromar los datos en el SELECT y a insertarlos en la tabla credit_cards
INSERT INTO `credit_cards` (id,user_id,iban,pan,pin,cvv,track1,track2,expiring_date)
SELECT id,user_id,iban,pan,pin,cvv,track1,track2,
STR_TO_DATE(expiring_date, '%m/%d/%y')
FROM credit_cards_raw;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:42:49	INSERT INTO `credit_cards` (id,user_id,iban,pan,pin,cvv,track1,track2,expiring_date) SELECT id,user_id,iban,pa...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.204 sec

Procedemos a importar e insertar los datos del archivo csv a la tabla **companies** sin transformaciones previas debido al buen estado de los datos

```
#Procedemos a importar e insertar los datos del archivo csv a la tabla companies
LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\Sprint 4 MODELAT SQL\\Archivos adjuntos sprint 4\\companies.csv'
INTO TABLE `companies`
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS ;

SELECT * FROM companies;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:03:35	SELECT * FROM companies	0 row(s) returned	0.000 sec / 0.000 sec
2	20:03:42	LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\S...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.015 sec

Procedemos a importar e insertar los datos del archivo csv a la tabla **products_raw** y luego se procederá a transformar el en select con la tabla **products**

```
#Procedemos a importar e insertar los datos del archivo csv a la tabla products_raw
LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\Sprint 4 MODELAT SQL\\Archivos adjuntos sprint 4\\products.csv'
INTO TABLE `products_raw`
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS ;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:18:18	LOAD DATA LOCAL INFILE 'C:\\Users\\aleja\\Documents\\Analisis de Datos\\Especialidad IT ACADEMY\\Sp...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.000 sec

Se procede a transfromar los datos en el SELECT y a insertarlos en la tabla **products**, en este caso solo hemos transformado el atributo price quitándole el símbolo de moneda.

```sql
-- Se procede a transfromar los datos en el SELECT y a insertarlos en la tabla products
INSERT INTO `products` (id,product_name,price,colour,weight,warehouse_id)
SELECT id,product_name,
REPLACE(REPLACE(price,'$',''), ',', '')
,colour,weight,warehouse_id
FROM products_raw;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:22:51	INSERT INTO `products` (id,product_name,price,colour,weight,warehouse_id) SELECT id,product_name, REPLA...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0	0.016 sec

Se procede a transformar, limpiar y a insertar en la tabla transactions con un SELECT desde la tabla **transactions_raw**

```sql
-- Se procede a transformar, limpiar y a insertar en la tabla transactions con un SELECT desde la tabla transactions_raw
INSERT INTO `transactions` (id,card_id,company_id,timestamp,amount,declined,user_id,lat,longitude)
SELECT id,card_id,business_id,timestamp,
CAST(REPLACE(REPLACE(amount,'$',''), ',', '') AS DECIMAL(10,2)), -- aplicamos esta conversion por si hubiese algun dato con moneda $ y con coma (,)
CAST(declined as unsigned), -- unsigned: entero positivo que actuara como booleano
CAST(user_id as unsigned),
lat,longitude
FROM transactions_raw;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:01:09	INSERT INTO `transactions` (id,card_id,company_id,timestamp,amount,declined,user_id,lat,longitude) SELECT id,...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	4.781 sec

Se procede a insertar los datos a **transactions_products** a través de un SELECT desde la tabla **transactions_raw**.

Se ha tenido que utilizar la **JSON_TABLE** para poder asignar una fila a cada **product_id**, pero antes hemos tenido que limpiar los datos y convertirlos en tipo de dato **JSON**, por lo que se ha tenido que eliminar los espacios dentro de la celda con un **REPLACE** y luego se ha tenido que agregar los corchetes necesarios mediante la función **CONCAT** para que los datos estén aptos para usar la función **JSON**.

```sql
-- Se procede a insertar los datos a transactions_products a traves de un SELECT desde la tabla transactions_raw
INSERT INTO `transaction_products`(transaction_id,product_id)
SELECT t.id, jt.product_id
FROM transactions_raw t,
JSON_TABLE(
    CONCAT('[',REPLACE(product_ids, ' ',''),']'), '$[*]'
    COLUMNS(
            product_id int PATH '$')
) as jt
WHERE t.product_ids IS NOT NULL  -- con estos ultimos dos filtros evitamos que se rompa el insert por un nulo o dato dañado
AND t.product_ids <> '';
```

Output

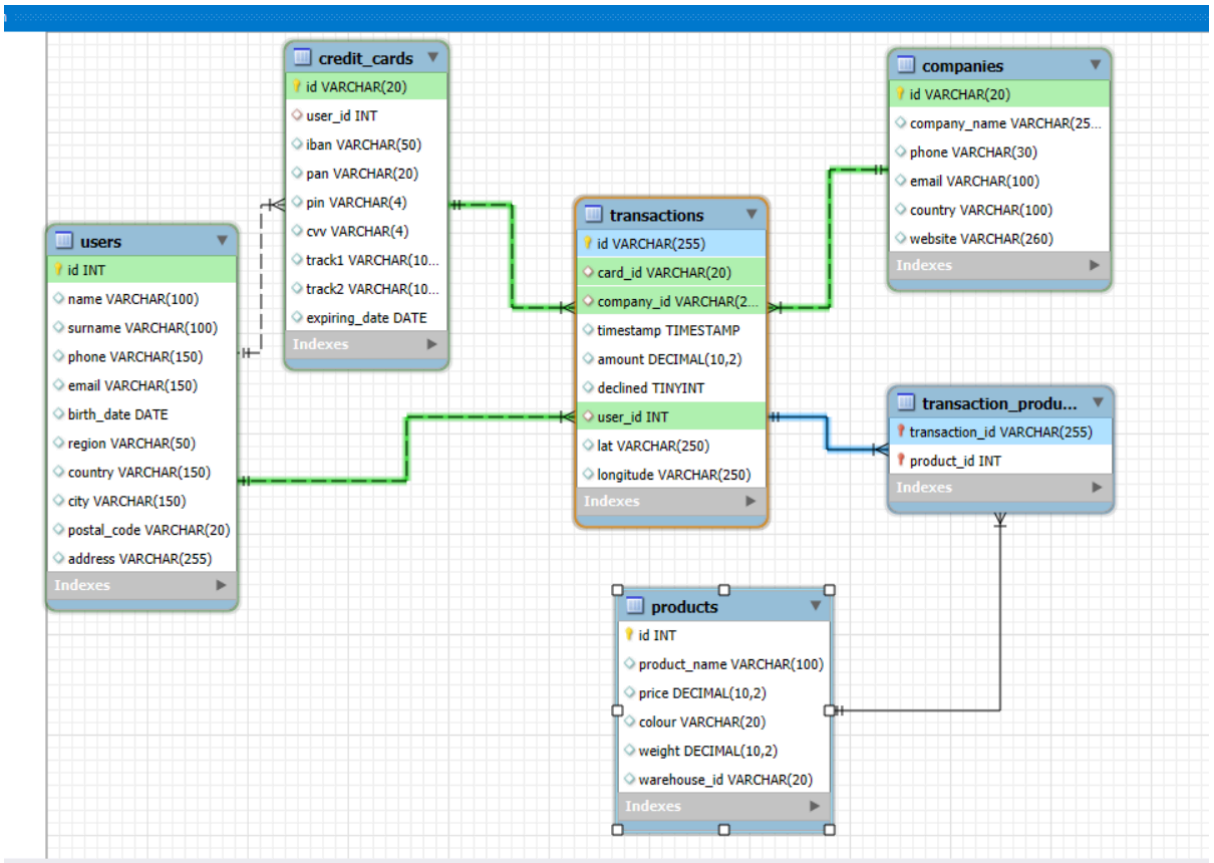
Action Output

#	Time	Action	Message	Duration / Fetch
1	12:28:50	INSERT INTO `transaction_products`(transaction_id,product_id) SELECT t.id, jt.product_id FROM transactions_ra...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0	6.907 sec

Una vez terminada la creación de las tablas:

transactions, credit_cards, users, companies, transaction_products y **products** sus relaciones y sus inserts con los datos obtenidos de los CSV obtenemos el siguiente diagrama:

DIAGRAMA FINAL OBTENIDO



En el diagrama se pueden observar 6 tablas resultantes relacionadas de forma que dan como resultado un modelo de esquema tipo ESTRELLA, donde la tabla **transactions** es la tabla de hechos conectadas a las demás a través de las foreigns keys. En un inicio se pensó que al crear la tabla intermedia **transaction_products** para resolver la relación muchos a muchos (N:M) entre la tablas **transactions** y **products** convertiríamos el quema ESTRELLA a tipo COPO DE NIEVE pero al no ser una subdivisión de ninguna de las tablas de dimensiones(**users,credit_cards,products,companies**) como tal sino una **tabla puente** por lo que no normaliza ninguna dimensión y el esquema se mantiene en tipo ESTRELLA.

Se procede a realizar los ejercicios debido a que la base de datos creada **operations** ya tiene todas las tablas, relaciones y datos correctos.

Exercici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

Se ha realizado el ejercicio teniendo en cuenta el declined para evitar contar transacciones declinadas, agrupando por **id** también para evitar duplicados de nombres y filtrando con el **Having** cumpliendo la condición.

```
/*Exercici 1
Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.*/
-- Con subconsulta A CORREGIR:
SELECT u.id,u.name,u.surname
FROM users u
where u.id in (
SELECT t.user_id
FROM transactions t
where t.declined=0
group by t.user_id
having COUNT(t.id)>80
);
```

Result Grid

id	name	surname
185	Molly	Gilliam
289	Dxwgi	Hwcru
318	Bnyr	Astuw
NULL	NULL	NULL

users 40

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	13:36:56	SELECT u.id,u.name,u.surname FROM users u where u.id in (SELECT t.user_id FROM transactions t where t.d...	3 row(s) returned	0.188 sec / 0.000 sec

Exercici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.
Se realiza la consulta utilizando JOINS para unir las tablas necesarias y aplicando su respectivo GROUP BY pero antes filtrando con un WHERE según la compañía solicitada.

```
/*Exercici 2
Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.*/

SELECT cr.iban, ROUND(AVG(amount),2) as average_amount
FROM transactions t
JOIN credit_cards cr
ON t.card_id=cr.id
JOIN companies co
ON t.company_id=co.id
WHERE co.company_name='Donec Ltd' AND t.declined=0
GROUP BY cr.iban
ORDER BY average_amount DESC;
```

Result Grid

iban	average_amount
XX3830178139196201993663 52	680.69
XX6377063573975703949739 13	680.01
XX9713939714652922023122 59	645.46
XX1718471169288923759693 07	628.89
XX2254246388185424062235 75	608.68
XX7488907290571957117660 71	607.29

Result 14

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	17:56:25	SELECT cr.iban, ROUND(AVG(amount),2) as average_amount FROM transactions t JOIN credit_cards cr ON t.c...	370 row(s) returned	0.015 sec / 0.000 sec

Nivell 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

Se procede a crear la tabla con nombre **card_status** donde se tendrán los atributos card_id y el estado de la tarjeta(sea activa o inactiva).

Se utiliza el comando CASE con un condicional WHERE para asignar el estado de la tarjeta fila por fila y obteniendo las últimas 3 transacciones por el comando ROW_NUMBER() y ordenando por timestamp DESC.

```
-- Creamos la tabla card_status

CREATE TABLE IF NOT EXISTS card_status
SELECT card_id,
CASE
    WHEN SUM(declined)=COUNT(*) THEN 'Inactive'-- Igualando el sum con el count se evita marcar como activa una tarjeta q tenga solo 2 transacciones y a su vez declinadas
    ELSE 'Active'
END AS current_status
FROM( SELECT card_id, declined,
 ROW_NUMBER() OVER(PARTITION BY card_id
 ORDER BY timestamp DESC) AS rn -- alias comun rn para la funcion ROW_NUMBER()
 FROM transactions) AS ranked
WHERE rn<=3
GROUP BY card_id;
-- COUNT(*) cuenta cuántas filas reales tiene esa tarjeta después del filtro, sea 1, 2 o 3.
-- Y SUM(declined) cuenta cuántas fueron declinadas. Si son iguales, todas fueron declinadas.
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:16:39	CREATE TABLE IF NOT EXISTS card_status SELECT card_id, CASE WHEN SUM(declined)=COUNT(*) THEN ...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.422 sec

Exercici 1

Quantes targetes estan actives?

```
-- Exercici 1
-- Quantes targetes estan actives?
SELECT COUNT(*) AS active_cards
FROM card_status
WHERE current_status='Active';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

active_cards
4995

Result 4

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:27:00	SELECT COUNT(*) AS active_cards FROM card_status WHERE current_status='Active'	1 row(s) returned	0.000 sec / 0.000 sec

Nivell 3

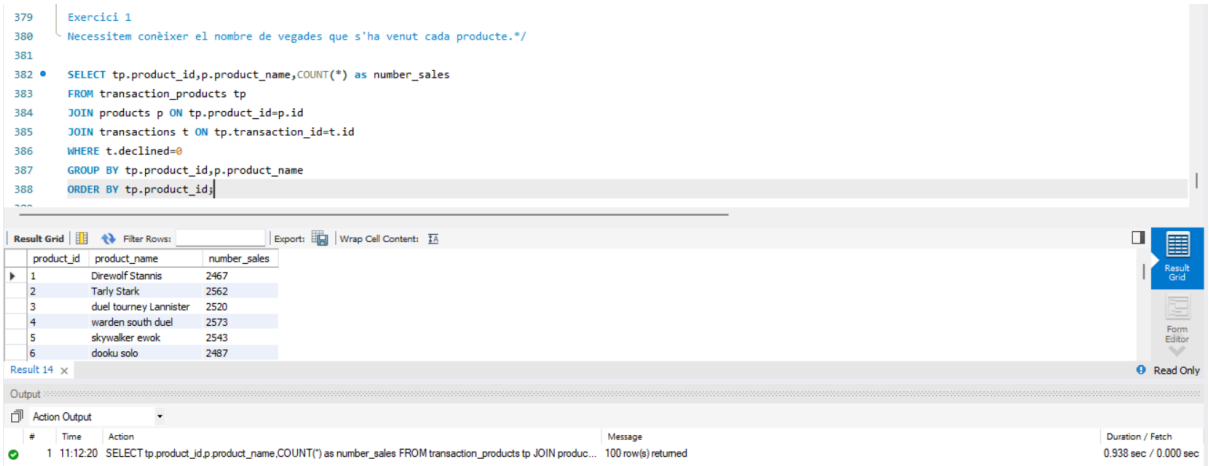
Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

Exercici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

Realizamos la consulta utilizando la tabla intermedia **transaction_products** creada para gestionar de forma correcta la relación muchos a muchos (N:M) que existía entre las tablas **transactions** y **products.**

Se realiza un JOIN entre la tabla **transaction_products** y **products** para obtener el nombre del producto y adicionalmente un JOIN con la tabla **transaction** para filtrar aquellas transacciones no rechazadas (declined=0).



```
Exercici 1
Necessitem conèixer el nombre de vegades que s'ha venut cada producte.*/

SELECT tp.product_id,p.product_name,COUNT(*) as number_sales
FROM transaction_products tp
JOIN products p ON tp.product_id=p.id
JOIN transactions t ON tp.transaction_id=t.id
WHERE t.declined=0
GROUP BY tp.product_id,p.product_name
ORDER BY tp.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	number_sales
1	Direwolf Stannis	2467
2	Tarly Stark	2562
3	duel tourney Lannister	2520
4	warden south duel	2573
5	skywalker ewok	2543
6	dooku solo	2487

Result 14 | Read Only

Output | Action Output

#	Time	Action	Message	Duration / Fetch
1	11:12:20	SELECT tp.product_id,p.product_name,COUNT(*) as number_sales FROM transaction_products tp JOIN produc...	100 row(s) returned	0.938 sec / 0.000 sec